



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I | Project Problem Statement Submission Form

Section / Team Details

Section No.:

7

Team Name:

Tiramisu

Project Title:

Alarm Clock — Linux Timer-Based Process Scheduling and Notification System

Team Members

Roll Number	Student Name	Signature
2520030237	Hansika Reddy Gunapati	
2520030048	Teja Hasini Reddy	
2520030064	Thanam Anthony Nissy	
2520030166	Simran Jangid	

Project Snapshot

Category	Platform	Language	Duration
Systems Programming	Linux / Ubuntu	C (POSIX APIs)	1 Term

Core OS Concepts Covered

Process Scheduling	Signal Handling	POSIX Threads	Synchronisation	Timers / Interrupts
--------------------	-----------------	---------------	-----------------	---------------------

1. Abstract

The Alarm Clock project is a Linux-based systems programming application that demonstrates efficient timer management and process scheduling using POSIX APIs. Traditional busy-waiting implementations of alarms waste CPU cycles by continuously polling the clock to check whether the target time has arrived, which is inefficient and wasteful of processor resources in a real operating system, since it prevents the scheduler from allocating that time to other ready processes.

This project replaces that inefficient approach with an event-driven design built on POSIX timers, the SIGALRM signal, and condition variables, allowing processes and threads to sleep until their scheduled wake-up time and resume only when notified by the kernel. The system allows a user to register multiple alarms, each associated with a specific process or thread, and observe how the Linux kernel manages timer expiry, signal delivery, and thread wake-up without wasting processor time.

The implementation uses POSIX threads to simulate multiple independent "sleepers," mutexes and condition variables to coordinate safe access to the shared alarm list, and signal handling to demonstrate asynchronous notification between the kernel and user-space threads. Each alarm is stored in a sorted, mutex-protected list, and the thread responsible for the nearest expiry computes the exact remaining time before sleeping, ensuring accuracy without continuous CPU polling.

Timer accuracy and CPU utilisation are measured and compared against a naive busy-wait implementation using `/proc` statistics and `strace`, highlighting the performance benefit of kernel-assisted sleeping over polling. This comparative analysis forms the experimental backbone of the project, providing concrete, measurable evidence of why event-driven synchronisation is preferred in real operating systems.

The expected outcome is a fully working command-line alarm clock utility that accurately triggers multiple concurrent alarms at their specified times, logs wake-up events with timestamps, and provides a clear, hands-on demonstration of process scheduling, signal handling, and thread synchronisation concepts that are central to Operating Systems and Systems Programming.

Efficient timer management is a fundamental requirement in real-world systems such as embedded devices, servers, and scheduling daemons, where thousands of timed events must be handled without exhausting CPU resources — making this project directly applicable beyond the classroom setting.

Particular attention is given to avoiding race conditions and missed wake-ups when multiple threads access the shared alarm list simultaneously, ensuring the solution is both thread-safe and deadlock-free.

2. Problem Statement

Applications that need to perform time-delayed actions often resort to busy-waiting loops that repeatedly check the current time, wasting CPU cycles and reducing overall system throughput. This is a poor scheduling practice from an Operating Systems perspective, since it prevents the CPU from being allocated to other ready processes. The problem this project addresses is: how can a Linux process or set of threads be made to "sleep" efficiently until a specified alarm time, and be woken up reliably and accurately by the kernel, while correctly synchronising access to shared alarm data across multiple threads? The project is directly relevant to core Operating Systems and Systems Programming concepts including process/thread scheduling, timers, signal handling, and synchronisation primitives.

3. Objectives

1. Design and implement a Linux alarm clock utility that registers and triggers multiple concurrent alarms without busy-waiting.
2. Use POSIX timers (timer_create/setitimer) and the SIGALRM signal to achieve accurate, kernel-assisted wake-up notifications.
3. Use POSIX threads with mutexes and condition variables to manage a shared alarm list safely across concurrent "sleeper" threads.
4. Measure and compare CPU utilisation and timing accuracy of the event-driven design against a naive busy-wait implementation using /proc and strace.

4. Proposed Methodology

The system will be implemented in C on Ubuntu Linux. A main thread accepts alarm requests (wake-up time and message) from the user via a command-line interface and inserts them into a shared, mutex-protected alarm list sorted by expiry time. Worker threads created with pthread_create act as independent sleepers: each thread waits on a condition variable, computing the remaining time to the next alarm and calling nanosleep or waiting on a POSIX interval timer (setitimer) configured to deliver SIGALRM at expiry. When a timer expires, the registered signal handler sets a flag and signals the condition variable, waking the responsible thread, which then prints the alarm message and removes the entry from the list. The design will be built incrementally: first a single-alarm SIGALRM-based prototype, then extended to multiple concurrent alarms using a sorted linked list and per-alarm threads, and finally instrumented with performance logging (elapsed time vs. requested time, CPU usage from /proc/[pid]/stat) to evaluate accuracy and efficiency. GDB and strace will be used throughout to debug signal delivery and thread scheduling behaviour.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
fork() / pthread_create()	Create independent processes/threads to act as concurrent alarm sleepers
alarm() / setitimer() / timer_create()	Schedule kernel-assisted timer expiry for each alarm
SIGALRM & sigaction()	Deliver and handle asynchronous notification when a timer expires
nanosleep()	Efficiently suspend a thread until the next alarm is due
pthread_mutex_t / pthread_cond_t	Protect and coordinate access to the shared alarm list across threads
/proc/[pid]/stat, strace, gdb	Analyse CPU utilisation, timer accuracy, and debug signal/thread behaviour

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030237	Hansika Reddy Gunapati	Designed the alarm scheduling module using POSIX timers and signal handling (SIGALRM); implemented core alarm-registration logic
2520030048	Teja Hasini Reddy	Implemented the multithreaded countdown/notification engine using POSIX threads, mutexes and condition variables
2520030064	Thanam Anthony Nissy	Built the CLI interface, input validation, and logging/testing framework; documented test cases and results
2520030166	Simran Jangid	Handled system-level integration (/proc monitoring, timer accuracy analysis), README, and final report

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and demonstration environment
C (POSIX APIs)	Core implementation language for system-level programming
GDB, strace	Debugging signal delivery and thread scheduling
Make / GCC	Building and compiling the project
Git / GitHub	Version control and source code management

8. Expected Outcome

A fully functional, event-driven Linux alarm clock utility that allows a user to register multiple concurrent alarms and receive accurate, timely notifications without CPU-wasting busy-waits. The project will produce measurable evidence (timing logs and CPU-usage comparisons) demonstrating the efficiency of signal- and condition-variable-based waiting over polling, along with complete source code, a Makefile, README, test cases, and a project report suitable for demonstration and viva.

9. GitHub Repository

GitHub Repository Link: https://github.com/7Hansika7/Tiramisu_OSSP/tree/main/ForgeOS